

EVOP 2026

Introduction to Linux Pt. 2

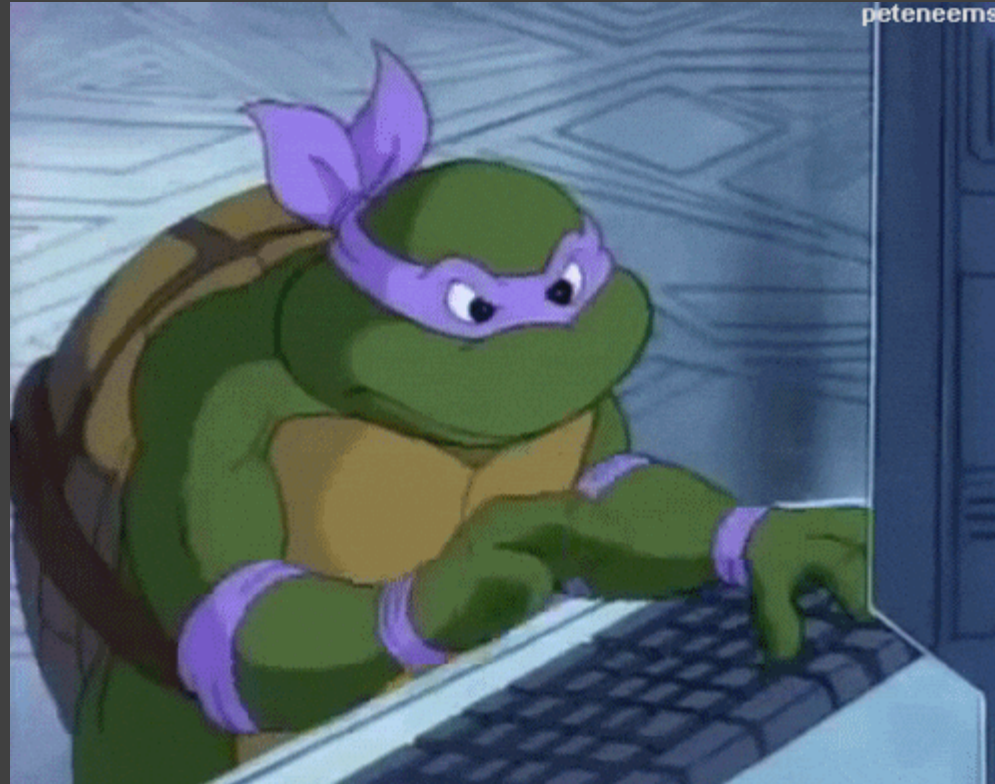
Dr. Isolde van Riemsdijk
Copenhagen University



KØBENHAVNS
UNIVERSITET



Moving from commands to scripts!



Regular expressions & repeating

And: &&

Or: ||

e.g. in grep or awk!

```
k00457149@andorra:~$ awk '$4 > 1 && $4 < 5 {print $1, $5-$4}' genes/chr8.gff | head
```

Exercise 13

Editing a file

Different file viewer programs (e.g. notepad-like):

nano

vim

vi

emacs

Editing a file

```
k00457149@andorra:~$ nano new_file.txt
```

```
GNU nano 7.2          new_file.txt *
I can write something here

some more

blabla

hahaha

```

^G Hilfe	^O Speichern	^W Wo ist	^K Ausschneiden	^T Ausführen	^C Position
^X Beenden	^R Datei öffnen	^I Ersetzen	^U Einfügen	^J Ausrichten	^/ Zu Zeile

Editing a file

```
k00457149@andorra:~$ nano new_file.txt
```

```
GNU nano 7.2          new_file.txt *  
  
I can write something here  
  
some more  
  
blabla  
  
hahaha  
  
Geänderten Puffer speichern?   
J Ja  
N Nein      ^C Abbrechen
```

A script (.sh)

```
#!/bin/bash
```

```
#SBATCH --job-name=iqtree  
#SBATCH --output=iqtree.out  
#SBATCH --error=iqtree.err  
#SBATCH --ntasks=1  
#SBATCH --cpus-per-task=4  
#SBATCH --mem-per-cpu=20G    # memory per cpu-core  
#SBATCH --time=10:00:00
```

```
## load iqtree  
module purge  
module load iqtree/2.4.0
```

```
## cd into directory with alignment  
cd /projects/alberdilab/people/lzh623/iqtree
```

```
## run iqtree to find best model  
iqtree -s forPhylo.collapsedPop.8x.diploid.filt.fasta -m TEST -msub nuclear -nt AUTO -pre model_knots
```

```
## when best model is found, perform a tree analyses
```


A bad script (.R) – myself 8 years ago :P

```
##Concatenate fasta files (after sequencer export)
library(seqinr)

setwd("G:/PhD Naturalis/In prep - Ommatotriton/Hendrix_nuclear/ABI files/KIAA_consensus_2")
getwd()
list.files()
samples<-c(list.files())
file_null<-data.frame()

for (i in 1:length(samples)){
  name<-read.fasta(file=samples[i])
  name<-paste(samples[i])
  name<-read.fasta(file=samples[i])
  file<-c(file_null, name)
  file_null<-file
}
file_null

write.fasta(file_null, names=samples, file.out="KIAA_alignment_2.fasta")

##To remove redundant taxa:
http://users-birc.au.dk/biopv/php/fabox/dnacollapser.php

##To convert fasta to phylip (doesnt work yet):
library("phylotools")

setwd("G:/PhD Naturalis/In prep - Ommatotriton/Hendrix_nuclear/Files Phase 2/KIAA")
tophylip<-read.fasta("KIAA_alignment_2_neat_unphased.fasta")
dat2phy(tophylip)

##To split fasta into seperate files using txt file
setwd("F:/PhD Naturalis/Transcriptome data/Bioinformatics")
original<-read.fasta("RefSeq_pipeline_2016_5_13.fasta", as.string=TRUE, set.attributes = FALSE)
name_list<-as.data.frame(read.table("list_refseq_uneven.txt"))
```

Using a Unix philosophy ...

Task: you want to automate downloading your reads (e.g. `wget`), doing a quality check on them (`fastqc`) and cleaning them (`fastp`)

How would you structure the script(s) for this task?



Exercise 14

Coffee break! (probably)

More concepts

A **loop**: you want to do something for each (set of) variables

```
viktor@viktor-VirtualBox:~$ cat dummy.sh
#!/bin/bash

echo "hello world"

for i in {1..5}
do
    echo "number: $i"
done
viktor@viktor-VirtualBox:~$ ./dummy.sh
hello world
number: 1
number: 2
number: 3
number: 4
number: 5
viktor@viktor-VirtualBox:~$ |
```

From: <https://linuxhint.com/bash-for-loop-one-line/>

More concepts

A **loop**: you want to do something for each (set of) files

```
#!/bin/bash

#SBATCH --job-name=downsample_reads
#SBATCH --output=downsample_reads.out
#SBATCH --error=downsample_reads.err
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=10
#SBATCH --mem-per-cpu=20G    # memory per cpu-core
#SBATCH --time=24:00:00

# run from software location:
cd /home/lzh623/data/Bombina_RNA/reads/

# load modules
module load seqtk/1.4

# take all the fastq.gz files here and subsample them 60% (=0.6), save them in the new folder
for fq in *.fq.gz; do seqtk sample -s 123 $fq 0.1 | gzip > ../reads_10/$fq; done
```

More concepts

A loop: you want to do something for each (set of) files

```
#!/bin/bash
#SBATCH --job-name=fastqc_bombi
#SBATCH --output=fastqc_bombi.out
#SBATCH --error=fastqc_bombi.err
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=1
#SBATCH --mem-per-cpu=1G    # memory per cpu-core
#SBATCH --time=72:00:00

# erase previous modules and load those necessary for fastqc
module purge
module load perl openjdk fastqc/0.12.1

# move into the directory where the read .fq.gz are located
cd /projects/alberdilab/people/lzh623/Bombina_RNA/filtered_reads/

# make a directory for the results in a new location
mkdir /projects/alberdilab/people/lzh623/Bombina_RNA/fastqc_filtered_results

# run fastqc
fastqc -o /projects/alberdilab/people/lzh623/Bombina_RNA/fastqc_filtered_results filtered_AHO*.fq.gz
```

More concepts

A loop: you want to do something for each **set of** files

```
# run fastp on data for all samples in directory
for r1 in EHI*_1.fq.gz; do
    r2=${r1/_1./_2.}
    fastp -i ${r1} -I ${r2} -o /projects/alberdilab/people/lzh623/Bombina_RNA/filtered_reads/filtered_${r1} -O /projec
done
```


More concepts

An array (just for fun)

```
#!/bin/bash
#SBATCH --job-name=sam_wgs
#SBATCH --output=sam_wgs_%a_%A.out
#SBATCH --error=sam_wgs_%a_%A.err
#SBATCH --array=1-24%5
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=20
#SBATCH --mem-per-cpu=6G      # memory per cpu-core
#SBATCH --time=24:00:00

## load bcftools
module purge
module load samtools/1.21

## cd into directory with sam files
cd /projects/alberdilab/people/lzh623/Bombina_wgs/bwa_sams/

## run samtools for all sams in another directory
sam=$(head -n $SLURM_ARRAY_TASK_ID file_list_sam.txt | tail -n 1)

## run samtools and pipe it to samtools sort directly
samtools view ${sam} -F3840 -bhS | samtools sort > ${sam}.bam
```

Exercise 15

The Advanced Stuff

BE AWARE: OPINIONS!

Environments >> preserve the versions of programs you are using

Conda >> a package manager that does this for you, across different operating system, mostly for python and R

Singularity and docker, are specific for High-Performance Computing environments (as opposed to Conda) >> reproducibility of science

Snakemake is a workflow manager: if you change something in your pipeline halfway, snakemake is only going to rerun from that point. (Terrible error reporting ... Unix philosophy?)

GitHub is a place where you can upload code, share (small) files, ... etc

slurm is a manager for job submissions (queue jobs and prioritizes jobs for efficiency)

GitHub

<https://docs.github.com/en/get-started/start-your-journey/hello-world>



RECAP



Common errors



```
cybergeek@cybergeek:~$ touch gfg_file(data).txt  
bash: syntax error near unexpected token `('
```

```
(base) [lzh623@mjolnirgate01fl Bombina_RNA]$ cat file_does_not_exist.txt  
cat: file_does_not_exist.txt: No such file or directory
```

Extra exercises!

